Практическая работа №14

Партицирование и наследование

Партиционирование—это метод разделения больших таблиц на более мелкие, управляемые части, называемые партициями. Этот подход позволяет улучшить производительность, упростить управление данными и повысить эффективность выполнения запросов.

Партицирование в первую очередь представлено в декларативном виде в виде разделения на секции. Можно выделить несколько возможных групп декларативного секционирования:

Партиционирование по диапазону (RangePartitioning):

Данные разбиваются на партиции по диапазонам значений. Например, таблица с данными о продажах может быть разбита по годам.

Пример:

CREATE TABLE sales (id SERIAL PRIMARY KEY, amount NUMERIC, sale_date DATE) **PARTITION BY RANGE (sale_date);**

CREATE TABLE sales_2022 PARTITION OF sales FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');

Партиционирование по списку (ListPartitioning):

Данные разбиваются по определенным значениям. Например, таблица с данными о клиентах может быть разбита по странам.

Пример:

CREATE TABLE customers (id SERIAL PRIMARY KEY, name TEXT, country TEXT) **PARTITION BY LIST (country);**

CREATE TABLE customers_usa PARTITION OF customers FOR VALUES IN ('USA');

CREATE TABLE customers_canada PARTITION OF customers FOR VALUES IN ('Canada');

Партиционирование по хэшу (HashPartitioning):

Данные распределяются по партициям на основе хэш-функции, что позволяет равномерно распределить записи.

Пример:CREATE TABLE orders (id SERIAL PRIMARY KEY, customer_id INTEGER) **PARTITION BY HASH (customer_id);**

CREATE TABLE orders_part1 PARTITION OF orders FOR VALUES WITH (MODULUS 4, REMAINDER 0);

CREATE TABLE orders_part2 PARTITION OF orders FOR VALUES WITH (MODULUS 4, REMAINDER 1);

Алгоритм создания секций:

1. Необходимо создать таблицу, из которой будут данные разбиваться на секции. Создается такая таблица с помощью добавления PARTITIONBY далее следует тип секционирования.
2. Создаются все соответствующие таблицы-секции с помощью добавления FORVALUES (…)
3. Данные импортируются в таблицу и автоматически распределяются по секциям.

```
create table car (region char(100), category char (100), parameter char(100), mode char(100), powertrain char(100),
vear integer. unit char(100). value decimal) PARTITION BY RANGE (vear):
```

Рисунок 1 – Создание таблицы с возможностью секционирования по диапазону

```
create table year_2019 partition of car for values from(2018) to (2019)
```

Рисунок 2 – Создание одной из таблиц секций

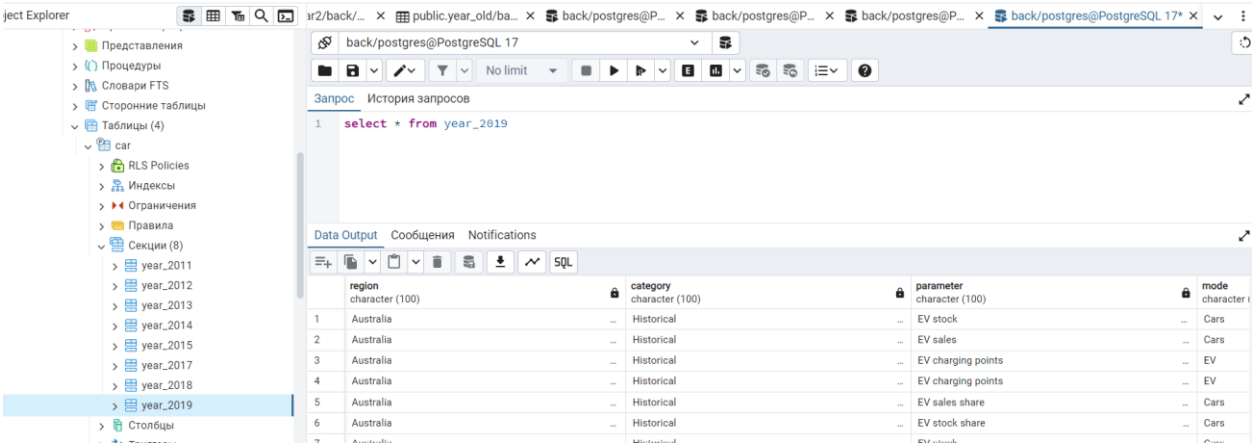


Рисунок 3 – Выборка из одной из таблиц секций и вид секционированных таблиц

```
select count(category), region
from car where year=2011 group by region order by region
```

Data Output | Сообщения | Notifications

	count bigint	region character (100)
1	4	Australia
2	6	Austria
3	32	Belgium
4	4	Brazil
5	14	Canada
6	6	Chile
7	27	Denmark
8	48	EU27
9	33	Europe

Рисунок 4 – Пример выполнения запроса с основной таблицей

Нельзя переделать обычную таблицу в секционированную или сделать секционированную таблицу обычной. Необходимо позаботиться об этом изначально. Ключевые значения могут вредить секционированию, поэтому рекомендуется их избегать.

Сами секции могут представлять собой секционируемые таблицы, благодаря применению так называемого *вложенного секционирования*. В секциях могут быть определены свои индексы, ограничения и значения по умолчанию, отличные от других секций. Индексы должны создаваться для каждой секции независимо.

Обычно набор секций, образованный изначально при создании таблиц, не предполагается сохранять неизменным. Чаще наоборот, планируется удалять старые секции данных и периодически добавлять новые. Одно из наиболее важных преимуществ секционирования состоит именно в том, что оно позволяет практически моментально выполнять трудоёмкие операции, изменяя структуру секций, а не физически перемещая большие объёмы данных.

Для того, чтобы обновить соответствующие данные необходимо воспользоваться процессом перепатицирования.

Процесс перепартицирования:

ATTACH PARTITION используется для добавления новой партиции к существующей таблице, которая уже настроена для партиционирования.

DETACH PARTITION в PostgreSQL используется для удаления партиции из партиционированной таблицы. При этом данные, содефржащиеся в партиции, остаются в базе данных, но партиция больше не будет частью родительской таблицы.

Наследование.

Вариантом секционирования в объектно-реляционной СУБД PostgreSQLможет также стать наследование. В таком случае таблица-потомок унаследует от таблицы-родителя столбцы и соответствующие строки. В отличие от декларативного секционирования, в таблицу потомок можно добавлять другие столбцы, которые нет в таблице-родителе, что запрещено в таблице-секции.

Использовать наследования для партицирования можно в тех случаях, когда набор столбцов партиции должен отличаться от основной таблицы, т.к. технологии наследования позволяют иметь у дочерней таблицы атрибуты отличные от родительской.

Запрос История запросов

```
1   create table region_A (check(region='Australia')) inherits (car2)
```

Рисунок 5 – Создание таблицы наследника.

Как видно из синтаксиса для наследника необходимо указывать таблицу-родителя после ключевого слово inherits, а также указывать ограничения в скобках у региона, чтобы обозначить наследуемые значения.

При создании наследуемой таблицы автоматически наследуются заданные строки, но не переносятся. В то время, как строки, к которым не был настроен доступ не будут доступны таблице-потомку.

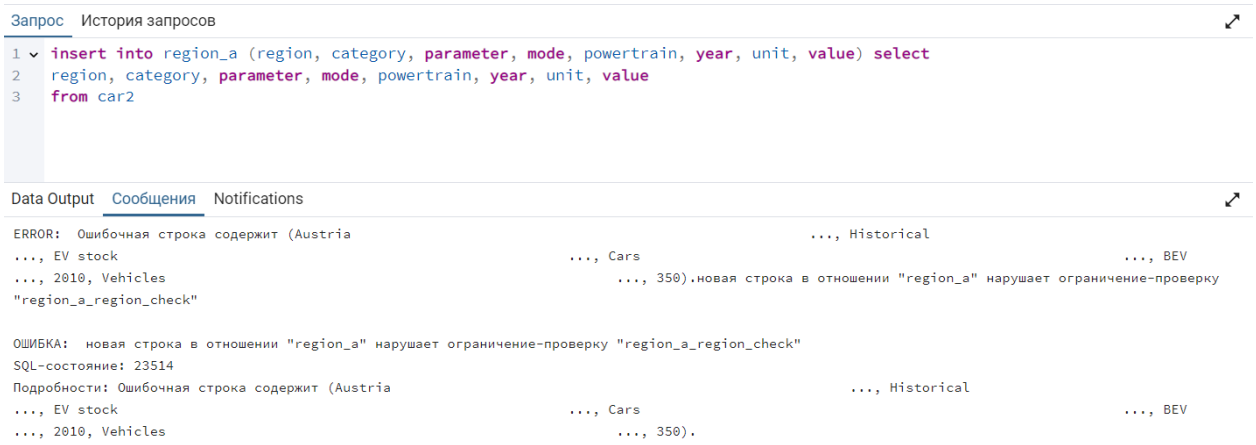


Рисунок 6 – Вставка данных в таблицу

На рисунке хорошо видно, что данные в таблицу не добавляются, т.к. нет доступа к соответствующим строкам, ограничение запрещает подобную вставку.

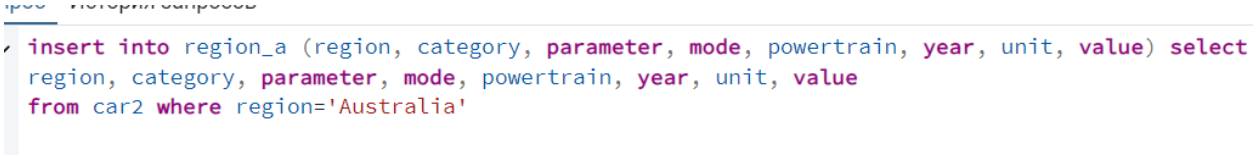


Рисунок 7 – Успешная вставка

В то же время, если ограничить строки необходимым условием, в наследуемую таблицу добавятся все необходимые строки.

Отсечение секций.

Для того, чтобы корректно был произведен поиск по секциям есть специальный параметр **enable_partition_pruning (boolean), который в**ключает или отключает в планировщике возможность отсекать секции секционированных таблиц из планов запроса. Также влияет на возможность планировщика генерировать планы запросов, позволяющие исполнителю пропускать (игнорировать) секции при выполнении запросов. По умолчанию имеет значение on (вкл.). Для того, чтобы убедиться в корректности работы секций можно с помощью знакомых операторов EXPLAINи ANALYZEпросмотреть, как будет работать планировщик в разных режимах данного параметра.

При активном отсечении секций видно, что секции, в которые не входит параметр не затрагиваются.

```
SET enable_partition_pruning = on; -- по умолчанию
explain analyze SELECT count(*), region FROM car WHERE year between 2012 and 2014 group by region;
```

Data Output | Сообщения | Notifications

	QUERY PLAN text
1	HashAggregate (cost=76.70..78.70 rows=200 width=109) (actual time=0.983..0.989 rows=15 loops=1)
2	Group Key: car.region
3	Batches: 1 Memory Usage: 40kB
4	-> Append (cost=0.00..73.36 rows=668 width=101) (actual time=0.045..0.505 rows=668 loops=1)
5	-> Seq Scan on year_2012 car_1 (cost=0.00..25.96 rows=264 width=101) (actual time=0.044..0.191 rows=264 loop...
6	Filter: ((year >= 2012) AND (year <= 2014))
7	-> Seq Scan on year_2013 car_2 (cost=0.00..30.37 rows=291 width=101) (actual time=0.013..0.165 rows=291 loop...
8	Filter: ((year >= 2012) AND (year <= 2014))
9	-> Seq Scan on year_2014 car_3 (cost=0.00..13.70 rows=113 width=101) (actual time=0.018..0.070 rows=113 loop...
10	Filter: ((year >= 2012) AND (year <= 2014))
11	Planning Time: 0.308 ms
12	Execution Time: 1.978 ms

Рисунок 8 – Сканирование таблицы с секциями с учетом отсечения

Запрос | История запросов

```
SET enable_partition_pruning = off; -- по умолчанию
explain analyze SELECT count(*), region FROM car WHERE year between 2012 and 2014 group by region;
```

Data Output | Сообщения | Notifications

	QUERY PLAN text
1	HashAggregate (cost=195.80..197.80 rows=200 width=109) (actual time=1.257..1.264 rows=15 loops=1)
2	Group Key: car.region
3	Batches: 1 Memory Usage: 40kB
4	-> Append (cost=0.00..192.44 rows=672 width=101) (actual time=0.031..0.867 rows=668 loops=1)
5	-> Seq Scan on year_2012 car_1 (cost=0.00..25.96 rows=264 width=101) (actual time=0.030..0.169 rows=264 loop...
6	Filter: ((year >= 2012) AND (year <= 2014))
7	-> Seq Scan on year_2013 car_2 (cost=0.00..30.37 rows=291 width=101) (actual time=0.015..0.147 rows=291 loop...
8	Filter: ((year >= 2012) AND (year <= 2014))
9	-> Seq Scan on year_2014 car_3 (cost=0.00..13.70 rows=113 width=101) (actual time=0.009..0.059 rows=113 loop...
10	Filter: ((year >= 2012) AND (year <= 2014))
11	-> Seq Scan on year_2015 car_4 (cost=0.00..32.71 rows=1 width=101) (actual time=0.125..0.125 rows=0 loops=1)
12	Filter: ((year >= 2012) AND (year <= 2014))
13	Rows Removed by Filter: 314
14	-> Seq Scan on year_2017 car_5 (cost=0.00..32.91 rows=1 width=101) (actual time=0.121..0.121 rows=0 loops=1)
15	Filter: ((year >= 2012) AND (year <= 2014))
16	Rows Removed by Filter: 327
17	-> Seq Scan on year_2018 car_6 (cost=0.00..16.20 rows=1 width=101) (actual time=0.058..0.058 rows=0 loops=1)
18	Filter: ((year >= 2012) AND (year <= 2014))

Total rows: 24 of 24 Query complete 00:00:00.097 Ln 1, Col 35

Рисунок 9 – Сканирование таблицы с секциями без учета отсечения

Задание.

1. Скачайте любой датасет на Kaggle.com/datasets. Создайте на основании его секционированную таблицу, в которой будут разбиты партиции по декларативному принципу.
2. Проведите несколько запросов, оцените время прохождения и его план в зависимости от включения и выключения секций.
3. Создайте на основании этого же датасета таблицу, в которой будут заданы потомки по другому принципу (отличному от ранее выбранного декларативного).
4. Проведите запросы на выборку в таблице-родителе и таблице-потомки, сравните их планы.